



Localization and Mapping

Course AIAA 4220

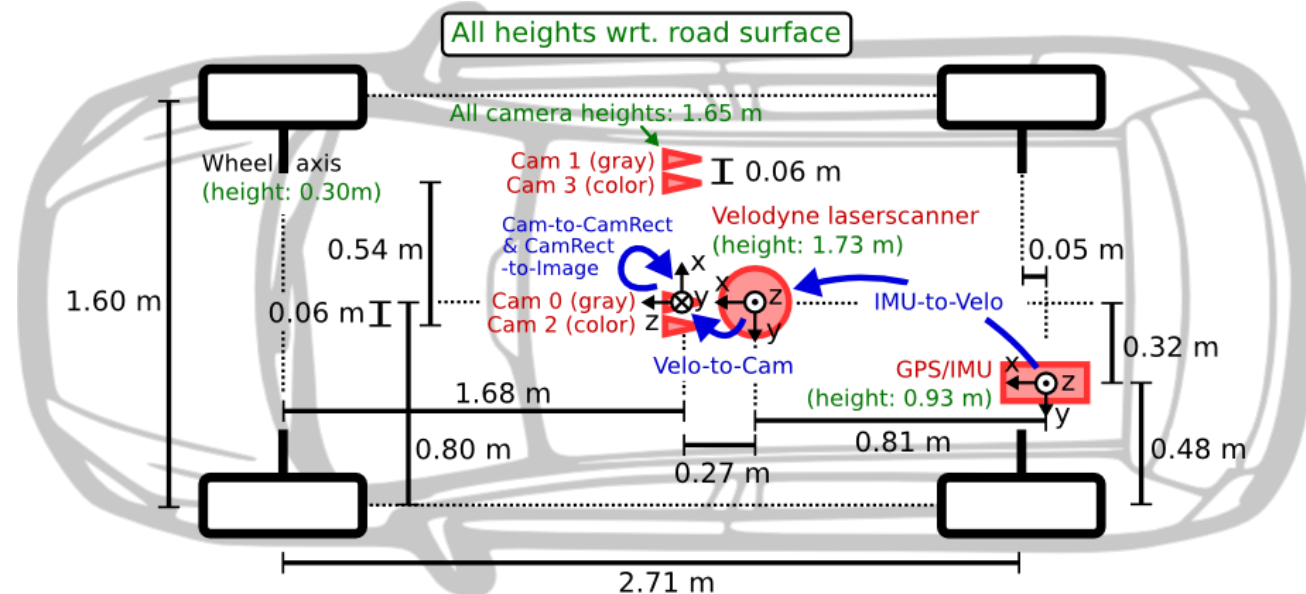
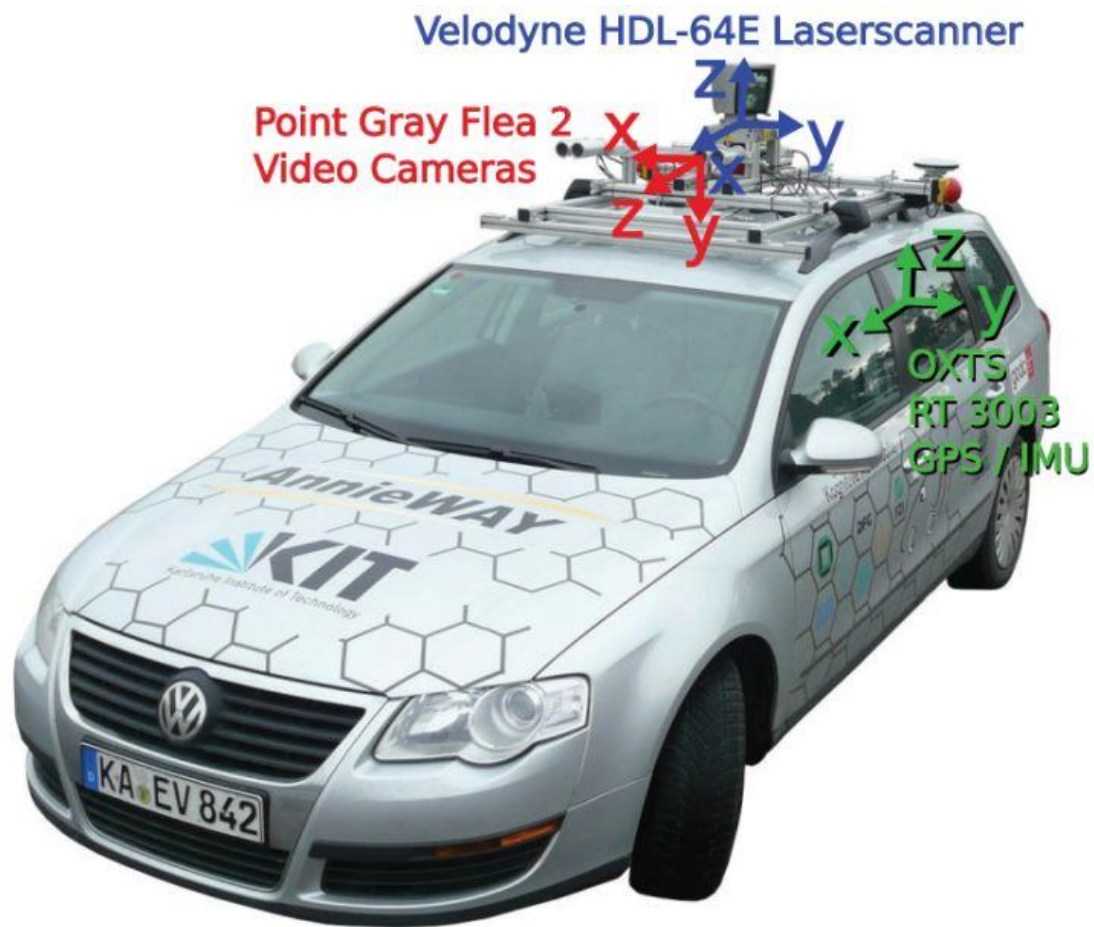
Week7- Lecture 13 & 14

Changhao Chen

Assistant Professor

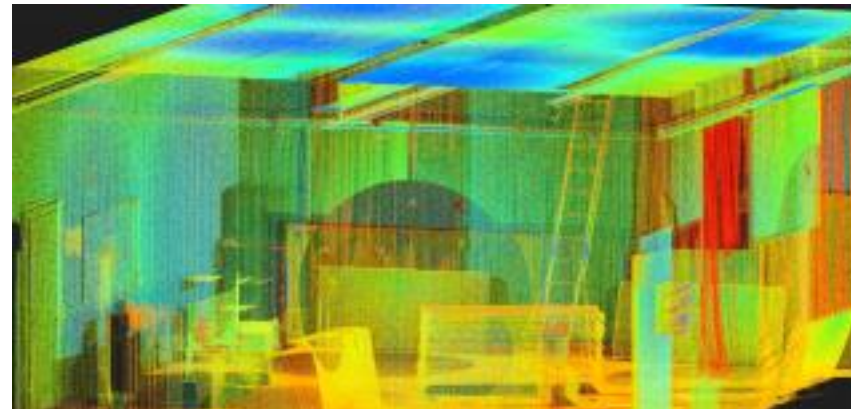
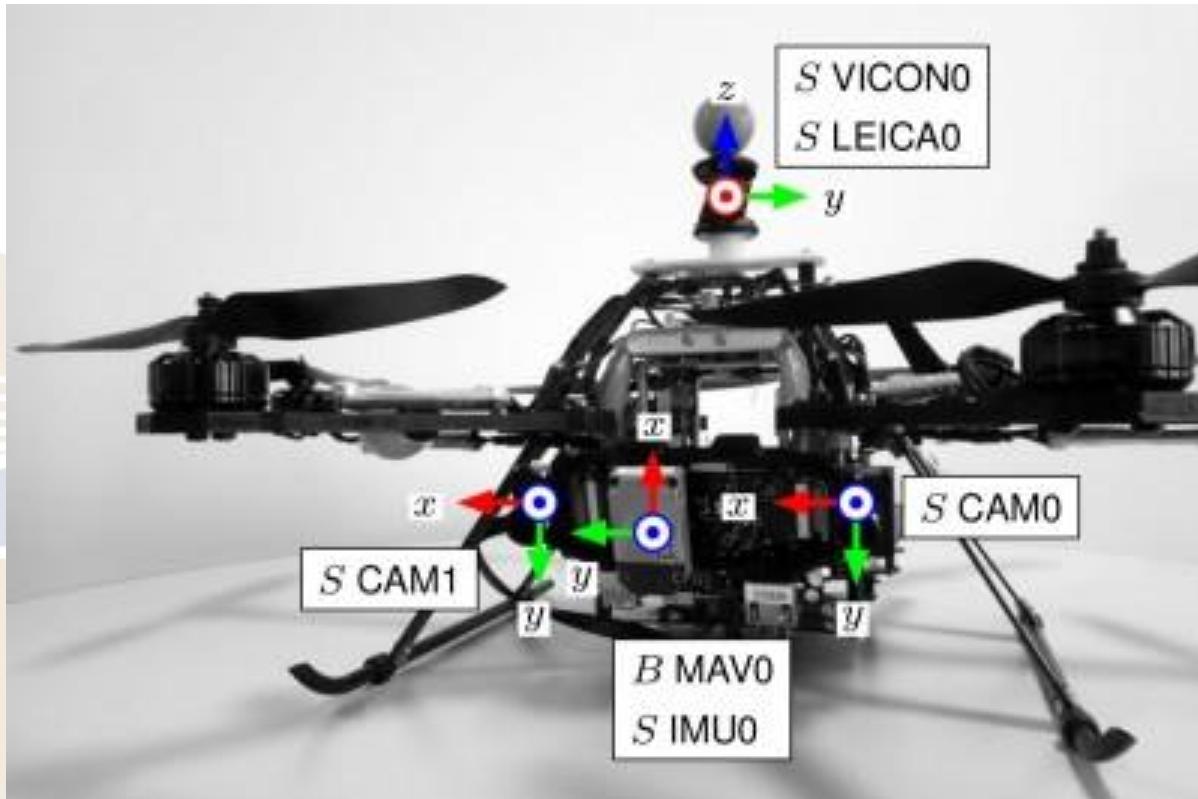
HKUST (GZ)

Typical Visual Navigation Platform: KITTI Dataset



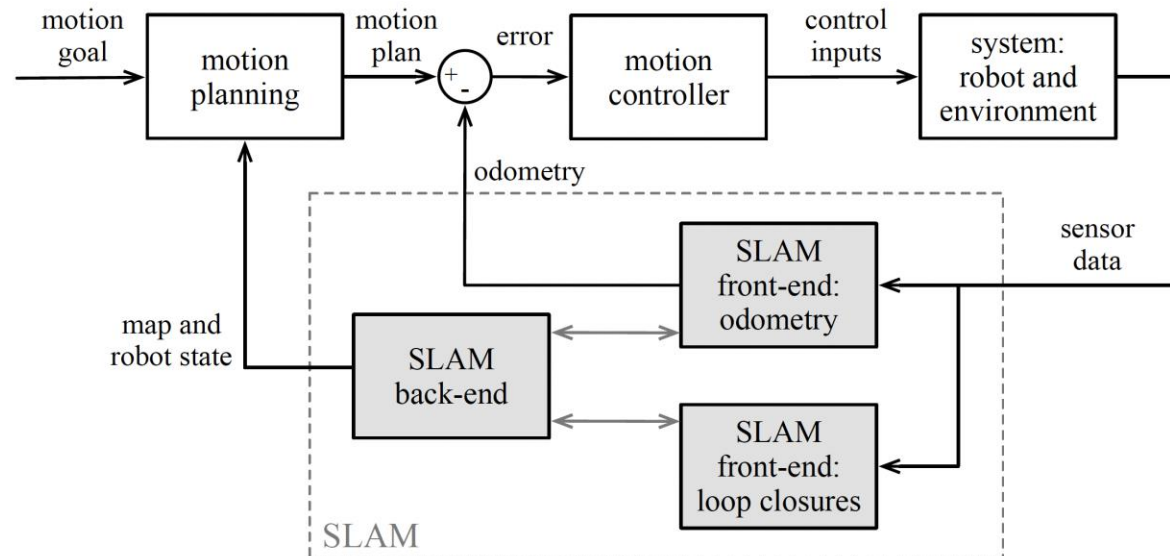
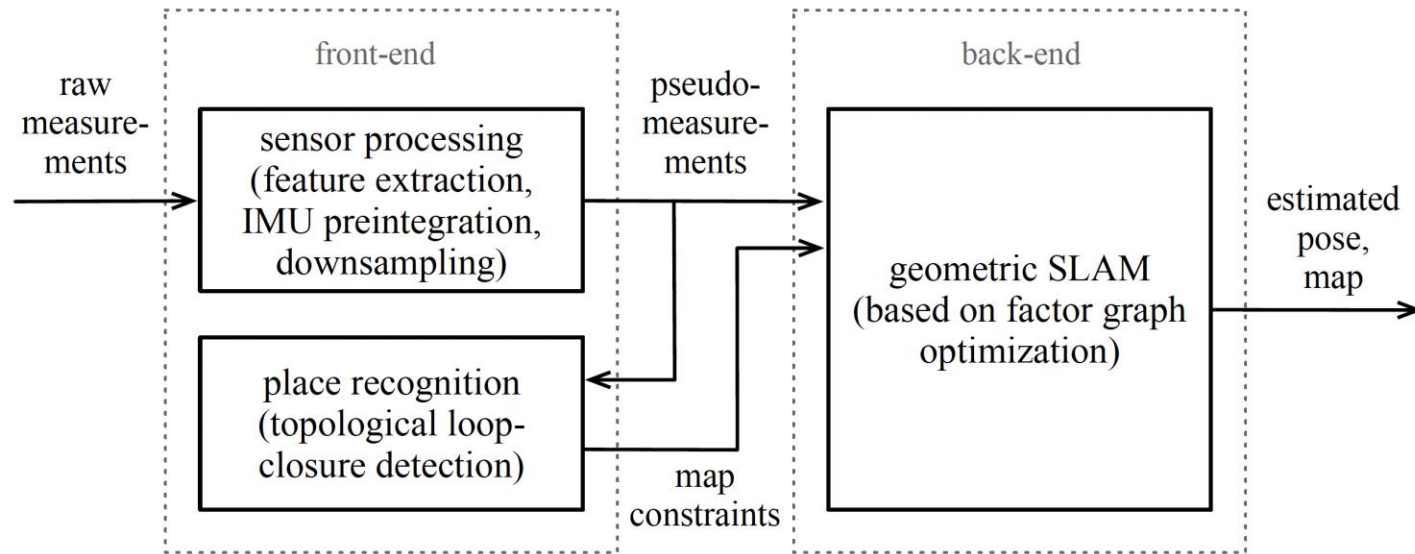
- 1 Inertial Navigation System (GPS/IMU): OXTS RT 3003
- 1 Laserscanner: Velodyne HDL-64E
- 2 Grayscale cameras, 1.4 Megapixels: Point Grey Flea 2 (FL2-14S3M-C)
- 2 Color cameras, 1.4 Megapixels: Point Grey Flea 2 (FL2-14S3C-C)
- 4 Varifocal lenses, 4-8 mm: Edmund Optics NT59-917

Typical Visual Navigation Platform: EuRoC Dataset

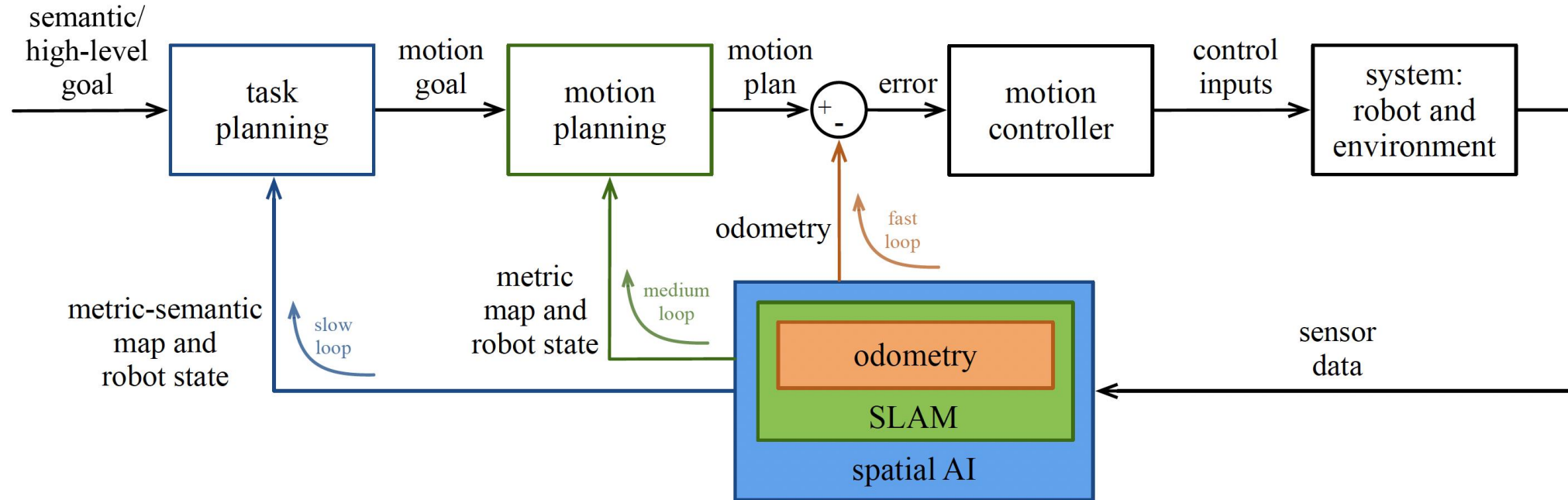


- Visual-Inertial Sensor Unit
 - **Stereo Images** (Aptina MT9V034 global shutter, WVGA monochrome, 2×20 FPS)
 - **MEMS IMU** (ADIS16448, angular rate and acceleration, 200 Hz)
 - Shutter-centric temporal alignment
- Ground-Truth
 - **Vicon** motion capture system (6D pose)
 - **Leica MS50** laser tracker (3D position)
 - Leica MS50 3D structure scan
- Calibration
 - Camera intrinsics
 - Camera-IMU extrinsics
 - Spatio-temporally aligned ground-truth

Localization and Mapping Problem



Localization and Mapping Problem



Rigid-body Transformations

We now generalize from pure rotation to full pose (rotation and translation).

Given:

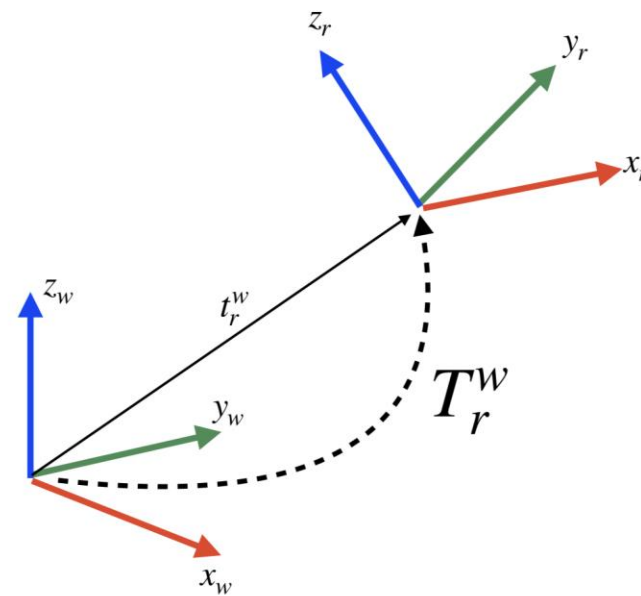
A point's coordinates in frame $\{r\}$: $\mathbf{p}^r$

The pose of $\{r\}$ relative to $\{w\}$, represented by the transformation matrix: $\mathbf{T}_r^w$

Compute: The point's coordinates in frame $\{w\}$: $\mathbf{p}^w$

The transformation is given by:

$$\mathbf{p}^w = \mathbf{R}_r^w \mathbf{p}^r + \mathbf{t}_r^w$$



Visual Pose Estimation

1. 2D-2D Pose Estimation (Monocular)

Input: Corresponding 2D points in two consecutive images.

Output: Relative camera rotation (R) and translation (t) up to scale.

Challenge: Scale ambiguity.

2. 3D-2D Pose Estimation (PnP)

Input: 2D image points and their corresponding known 3D points (from a map or previous reconstruction).

Output: Absolute 6-DOF camera pose (R , t).

Advantage: Provides metric scale.

2D-2D Correspondence

Input:

Two images of the same rigid scene, taken from two different camera positions.

A set of 2D-2D correspondences: $\{p_i \leftrightarrow p'_i\}$

$p_i = (u_i, v_i, 1)$ is a keypoint in Image 1 (in homogeneous coordinates).

p'_i is the matching keypoint in Image 2.

Output:

The relative pose (R, t) of Camera 2 relative to Camera 1.



The 2D-2D Pose Estimation

- 1) **Input:** Image pair + 2D-2D correspondences (from SIFT, ORB, etc.).
- 2) **Normalize Coordinates:** (Optional but recommended) Transform points using K^{-1} to work with E for better numerical stability.
- 3) **Robust Estimation:** Use the 8-Point Algorithm inside a RANSAC loop to compute the Essential Matrix E .
- 4) **Pose Extraction:** Perform SVD on E to get the four possible (R, t) solutions.
- 5) **Disambiguation:** Perform a Cheirality Check (triangulation) to find the single physically possible pose.
- 6) **Output:** The relative camera pose $T = [R \mid t]$, an element of $SE(3)$.

3D-2D Pose Estimation (PnP)

Question: "I have a pre-built map of the world. I see it with my camera. Where am I?"

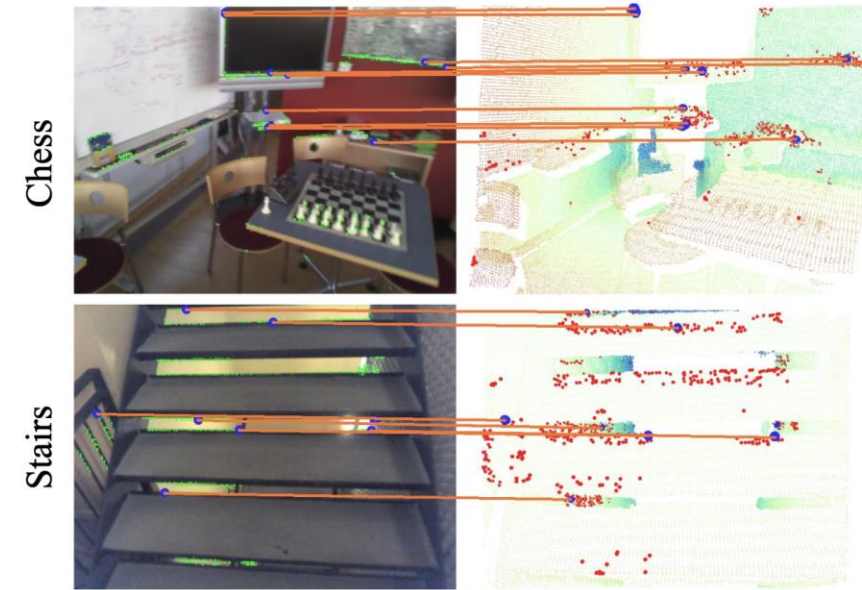
Input:

3D Map Points: Known 3D landmarks ($P_i = [X_i, Y_i, Z_i]^T$) from a prebuilt map or SLAM.

2D Image Points: Their current 2D projections ($p_i = [u_i, v_i]^T$) in the camera image.

Output: **The 6-DOF camera pose (Rotation R and Translation t) relative to the map.**

This is the Perspective-n-Point (PnP) problem.



3D-2D Pose Estimation (PnP)

Solves Scale: Unlike 2D-2D methods, PnP provides metric scale directly from the map.

Efficiency: Extremely fast pose estimation for each new camera frame.

Enables Localization: The core algorithm for matching a live camera feed to a pre-built HD Map.

Foundation for Tracking: Used in every major SLAM system (like ORB-SLAM) for tracking the camera's position in the local map.

3D-2D Pose Estimation (PnP)

The Pinhole Camera Model & The Goal

The projection of a 3D point P to a 2D pixel p is given by:

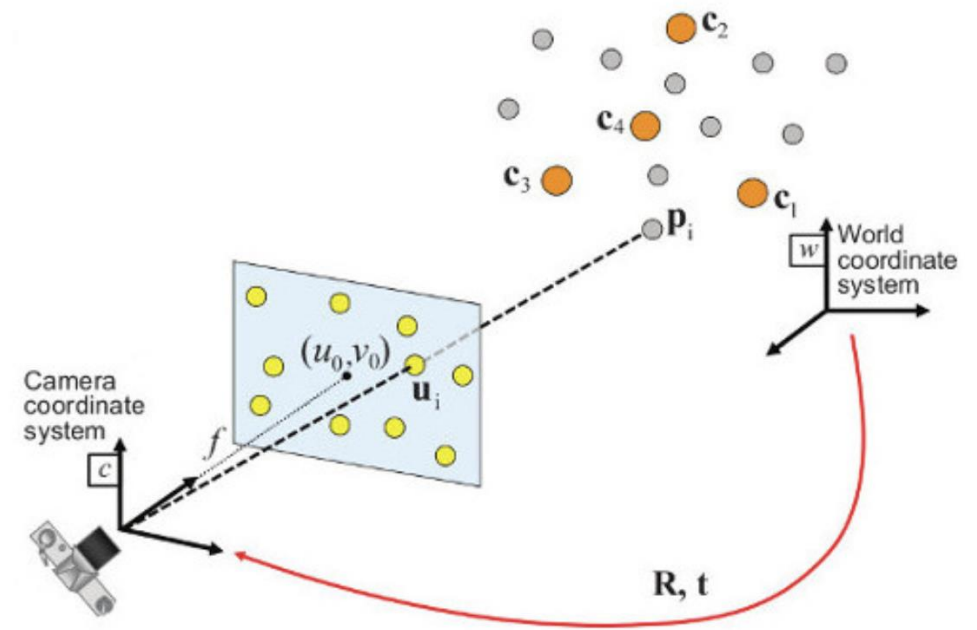
$$s \mathbf{p} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \mathbf{P}$$

where s is a scale factor (the point's depth).

Known: $\mathbf{K}$ (camera intrinsics), $\mathbf{P}$ (3D point), p (2D pixel).

Unknown: $\mathbf{R}$ (rotation), $\mathbf{t}$ (translation).

Our goal: Find the $\mathbf{R}$ and $\mathbf{t}$ that best satisfy this equation for all n point correspondences (P_i, p_i) .



VO and SLAM

Visual Odometry (VO)

Goal: Estimate the ego-motion of a vehicle incrementally from camera images.

Focus: Local consistency. Output: A locally accurate trajectory.

Weakness: Drift. Small errors accumulate over time, making the estimated path diverge from the true path.

Visual SLAM (Simultaneous Localization and Mapping)

Goal: Build a consistent global map of the environment while simultaneously localizing within it.

Focus: Global consistency. Output: A globally consistent map and trajectory.

Solution to Drift: Loop Closing - detecting previously visited locations and correcting the entire map.

Feature-Based SLAM

Visual Pose Estimation in SLAM System

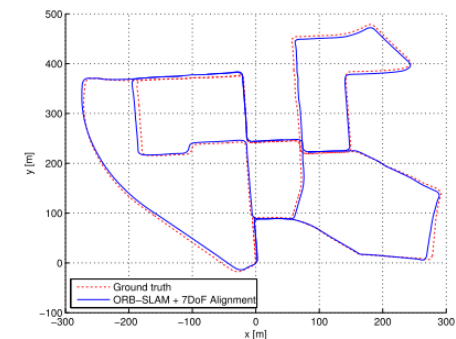
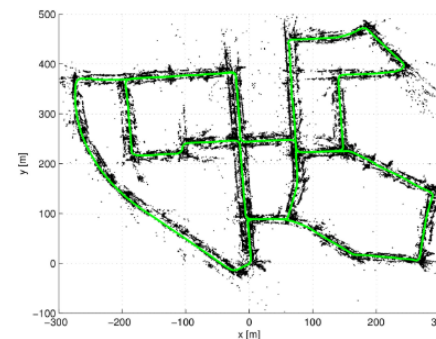
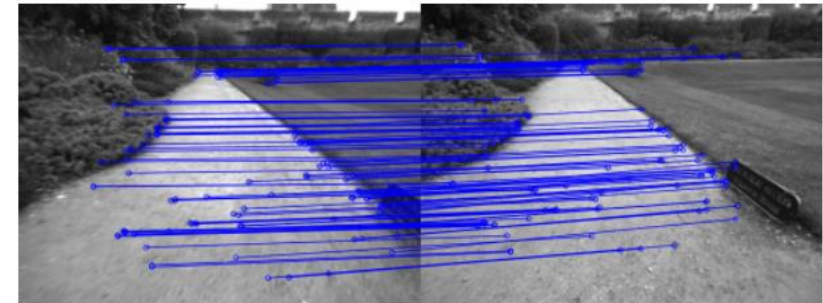
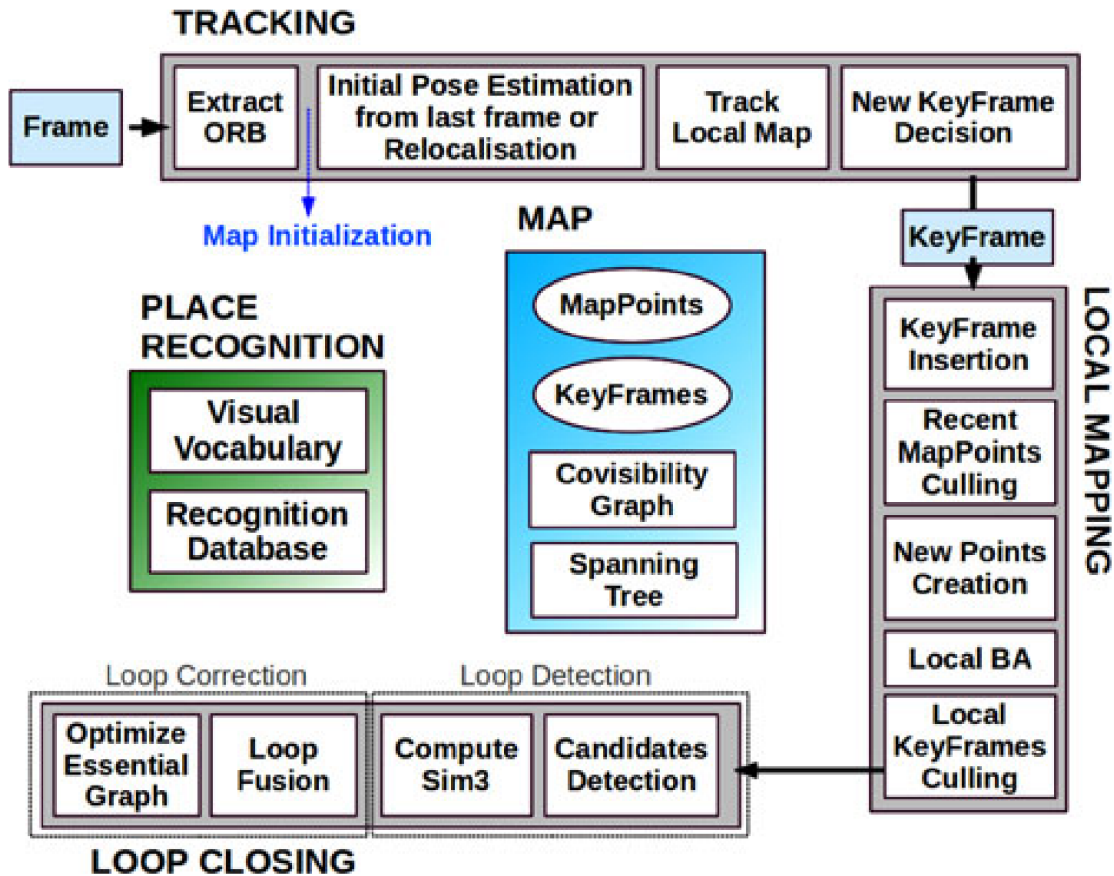
1. Initialization: Use 2D-2D (Epipolar) to bootstrap the map.
2. For every new frame:
3. Track features from previous frame or map.
4. Solve PnP (using RANSAC + EPnP) to get a robust, initial pose estimate.
5. (Optional) Pose-only Bundle Adjustment: Refine the current camera pose by minimizing reprojection error over all inliers.

This provides the real-time, robust localization for the vehicle.

Feature-Based Visual SLAM

ORB-SLAM : A Versatile and Accurate Monocular SLAM System (2015)

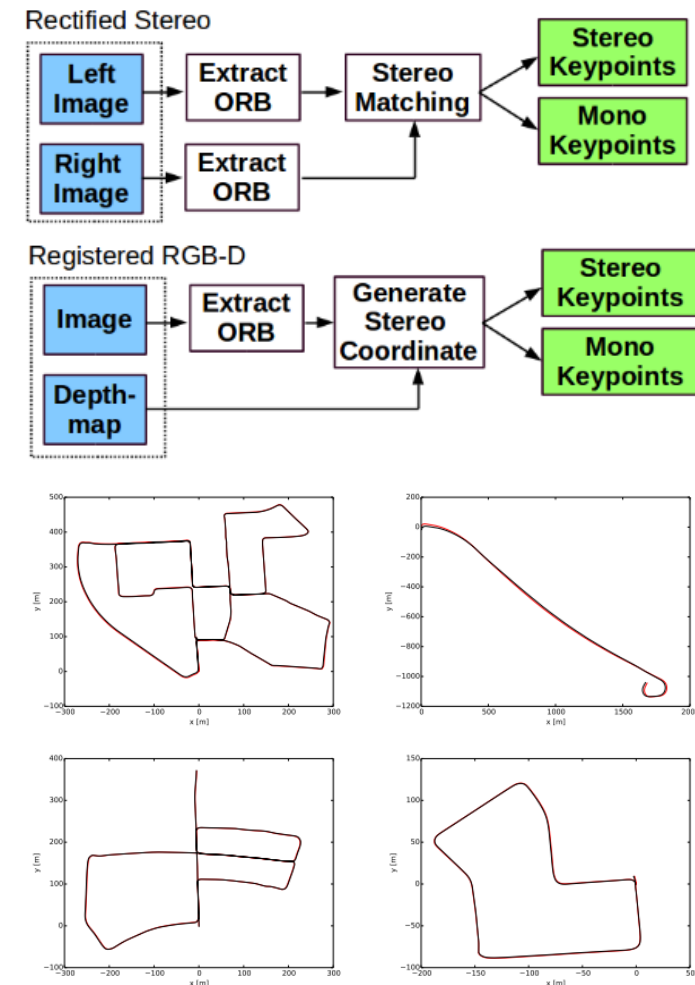
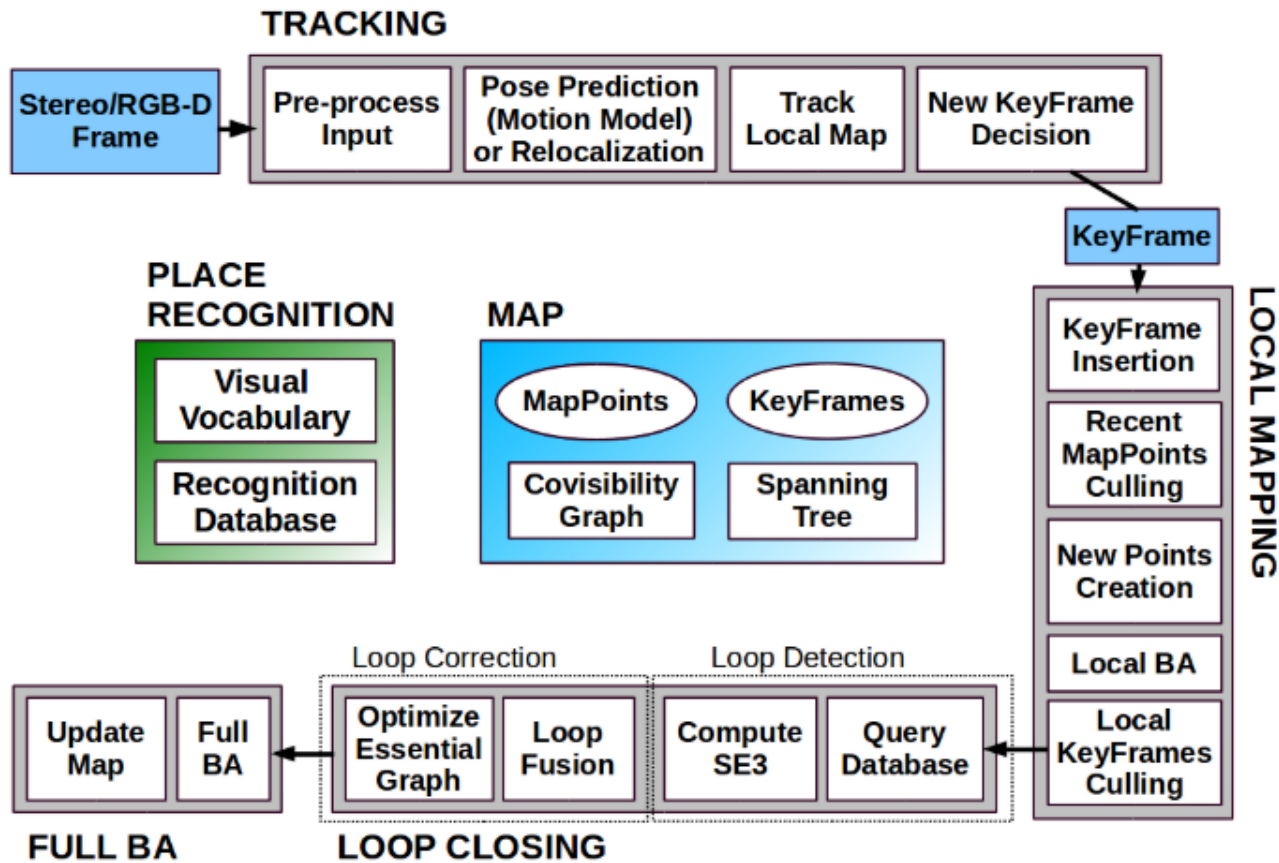
1. Tracking: feature extraction, pose estimation from last frame or global relocalization
2. Local mapping: key frame insertion, update map, local bundle adjustment
3. Loop Closing: loop-closure detection, graph optimization



Feature-Based Visual SLAM

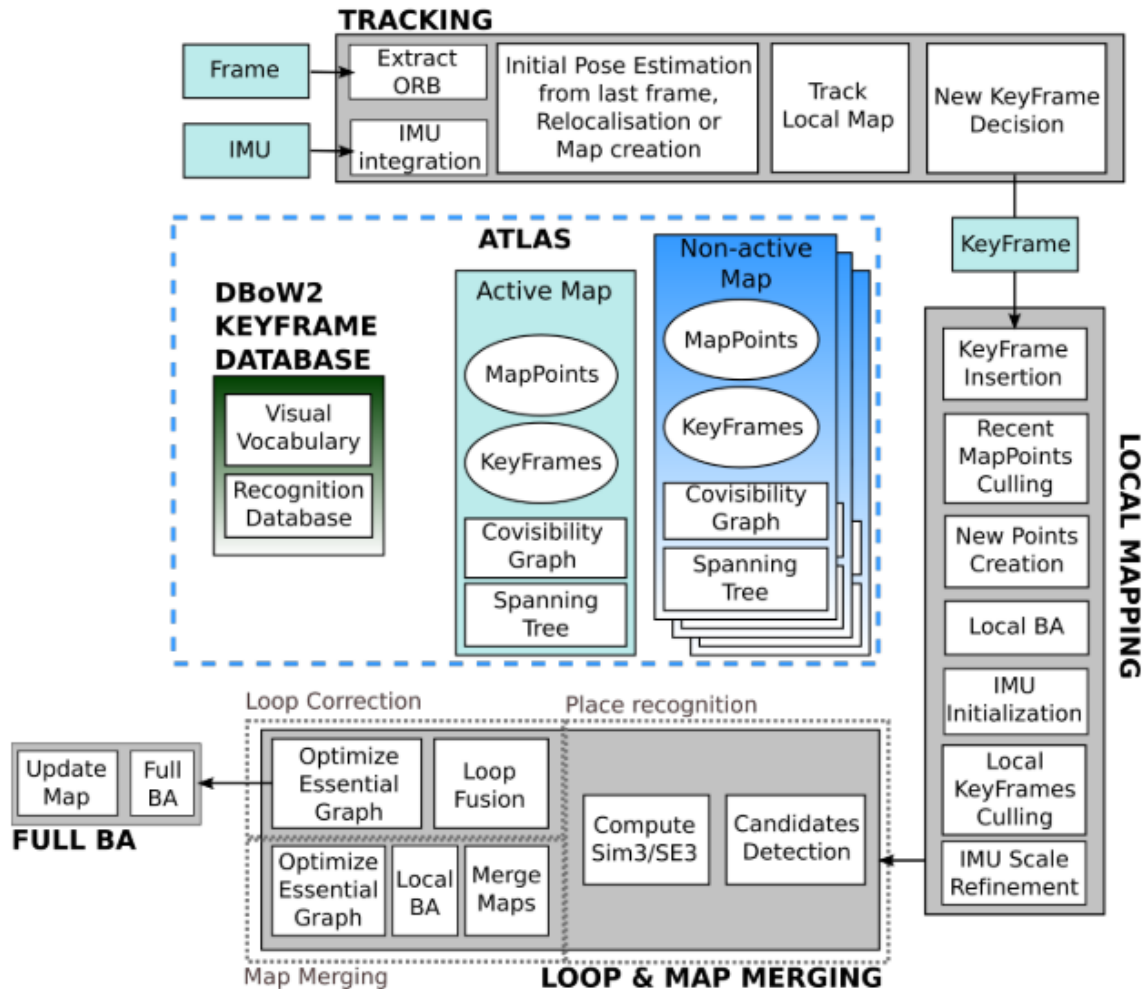
ORB-SLAM2 : an Open-Source SLAM System for Monocular , Stereo and RGB-D Cameras (2017)

Support monocular, stereo and RGB-D cameras



Feature-Based Visual SLAM

ORB-SLAM3 : An Accurate Open-Source Library for Visual , Visual-Inertial and Multi-Map SLAM (2020)



1. **Short-term data association**: matching map elements obtained during the last few seconds, e.g. VO
2. **Mid-term data association**: matching map elements that are close to the camera whose accumulated drift is still small, e.g. local BA
3. **Long-term data association**: matching observations with elements in previously visited areas using a place recognition technique, e.g. pose-graph optimization

- Multi-map
- MAP Estimation

Feature-Based Visual SLAM

- **Tracking (Localization)**: Estimate the camera pose for each new frame.
- **Mapping (Mapping)**: Estimate the 3D structure of the environment (points, landmarks).
- **Place Recognition (Loop Closing)**: Detect when the vehicle has returned to a previously visited area.
- **Global Optimization (Graph Optimization)**: Correct the accumulated drift by adjusting the entire map and trajectory.

Component 1: Tracking - The Frontend

Goal: Estimate the camera pose for every new frame.

How it works:

1. **Extract ORB features** from the current frame.
2. **Initialization:** If no map exists, use 2D-2D epipolar geometry to bootstrap the first pose and map.
3. **For subsequent frames:**
 - Motion Model: Predict the new pose from constant velocity assumption.
 - Track Local Map: Search for matches between current frame and local map points.
 - Pose Optimization: Use Motion-only Bundle Adjustment to refine the current camera pose by minimizing the reprojection error of the matched map points.
4. **Keyframe Decision:** Decide if the current frame is a keyframe (if tracking is weak, large change in viewpoint, etc.).

Component 1: Tracking - The Frontend

Problem: Processing every frame is computationally expensive and redundant.

Solution: Only process a selected subset of frames (Keyframes) in depth.

A Keyframe is chosen based on:

- Time passed since last keyframe.
- Large change in viewpoint (rotation/translation).
- Amount of new, unmatched features (tracking quality).

Keyframes store the full-resolution image, extracted features, and their pose.

Component 2: Local Mapping - The Middle-End

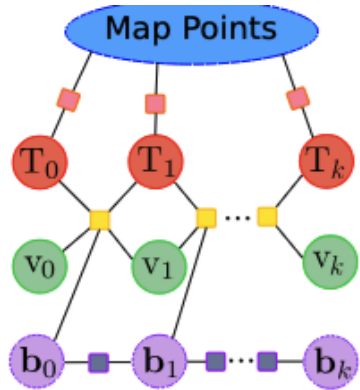
Goal: Refine the local map and create new 3D points.

Operates on new keyframes in a separate parallel thread.

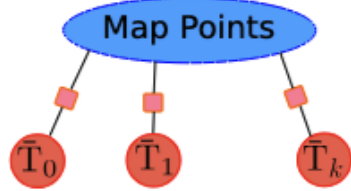
How it works:

1. **Keyframe Insertion**: Add the new keyframe to the map.
2. **Recent Map Point Culling**: Remove unstable map points that weren't found in many recent frames.
3. **New Map Point Creation**: Triangulate new 3D points from unmatched features between connected keyframes.
4. **Local Bundle Adjustment**: The core! Optimize the current keyframe, its covisibility keyframes, and all map points seen by them. This ensures local consistency.
5. **Local Keyframe Culling**: Remove redundant keyframes.

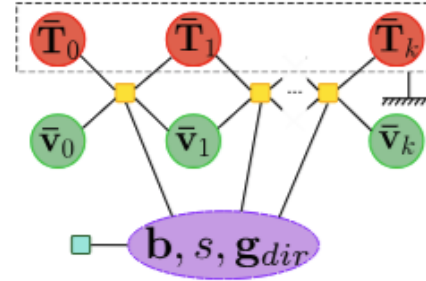
The Covisibility Graph



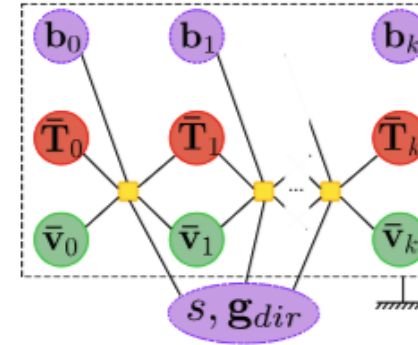
(a) Visual-Inertial



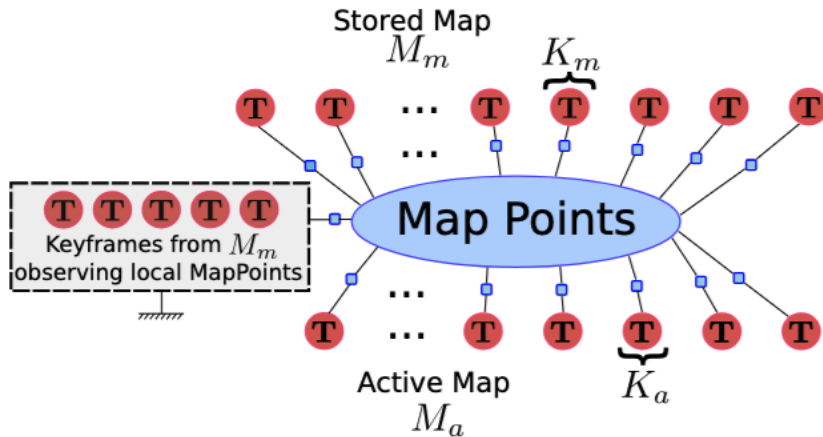
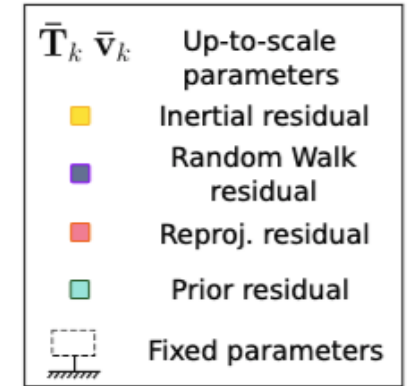
(b) Visual-Only



(c) Inertial-Only



(d) Scale and Gravity



(a) Visual welding BA

A powerful data structure that defines the relationship between keyframes.

Two keyframes are connected if they share a sufficient number of common map points.

The local map for a keyframe is defined by its covisibility keyframes.

This makes optimization efficient and scalable.

Component 3: Loop Closing - The Backend

Goal: Detect large loops and correct the accumulated drift globally.

Runs in a separate parallel thread.

1. **Loop Candidate Detection:** Use a Bag-of-Words (BoW) place recognition module to find a previous keyframe that looks similar to the current one (but is not a neighbor).
2. **Loop Validation:** Compute a similarity transform between the current and loop keyframe using RANSAC and PnP to geometrically verify the match.

Loop Correction:

- Fuse Duplicate Map Points: Merge the two instances of the same physical landmark.
- Essential Graph Optimization: Perform a pose-graph optimization over a spanning tree of the covisibility graph to correct the drift. This efficiently aligns the two sides of the loop.

3. **Full Bundle Adjustment (Optional):** Run a full global BA in a separate thread to get the optimal solution (very slow, but can run in the background).

Component 4: Relocalization

What if tracking is lost? (e.g., due to a sudden motion, occlusion, or a bright light).

ORB-SLAM doesn't just give up.

It compares the current frame to all previous keyframes in the map using the Bag-of-Words model.

If a match is found, it computes the camera pose via PnP and continues tracking.

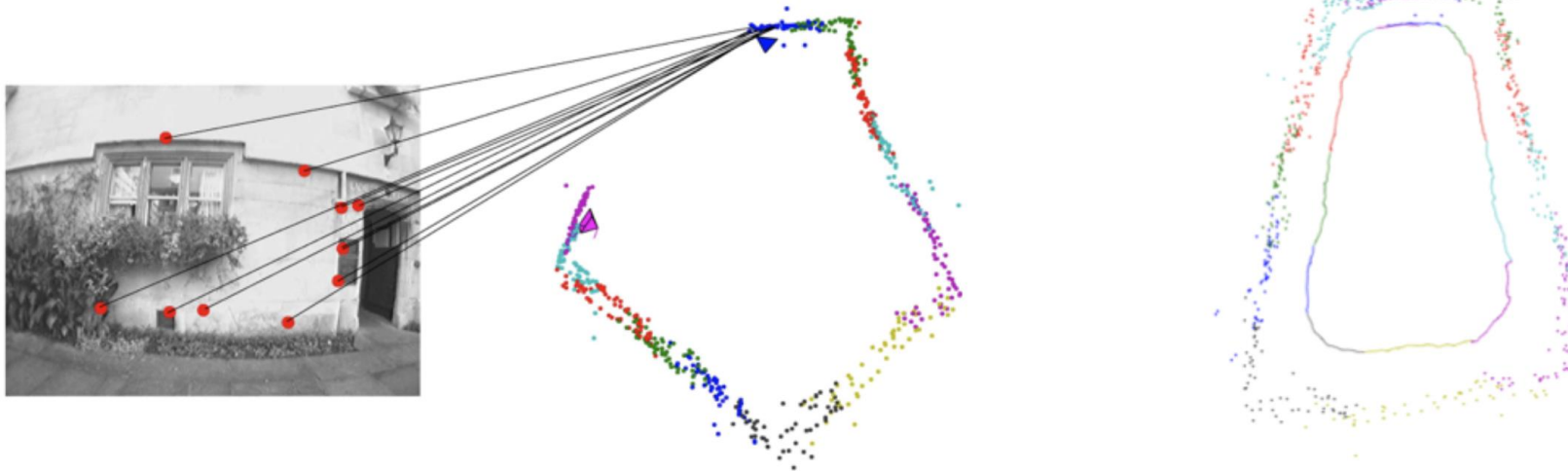
This makes the system incredibly robust.

Loop Closing

Loop Closing - The process of recognizing a previously visited location and correcting the accumulated drift.

Functions:

- **Drift Correction:** Significantly reduces long-term error.
- **Map Consistency:** Produces a globally consistent map.
- **Enables Long-Term Autonomy:** A vehicle can operate for hours/days without getting lost.



Place Recognition

Definition: The task of determining where an image was taken by matching it against a database of geo-referenced images.

It's not just image retrieval! It's about *appearance-invariant* recognition.

Input: A query image from the vehicle's current view.

Output: A binary decision ("Is this a loop?") and/or a match to a previous location in the map.



(a)



(b)

https://blog.csdn.net/weixin_44832149

Why is it Challenging for Intelligent Vehicles?

Viewpoint Change: The same place looks different when approached from a different direction.

Condition Change (Perceptual Aliasing):

Time of Day: Morning vs. Night.

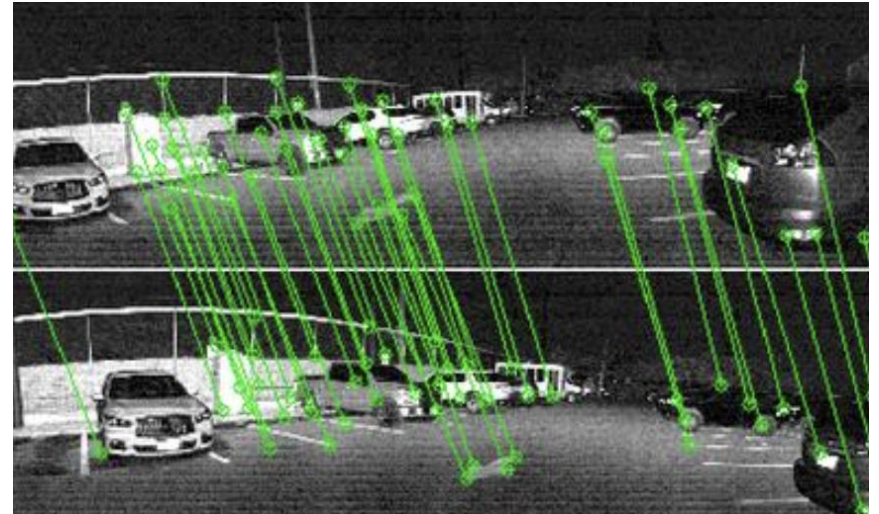
Weather: Sunny vs. Rainy vs. Snowy.

Seasons: Summer vs. Winter.

Dynamic Objects: Cars, pedestrians, which are not part of the "place."

Structural Changes: Construction, new buildings.

Scale & Speed: Vehicles move faster than robots, requiring efficient algorithms.



Bag-of-Words (BoW)

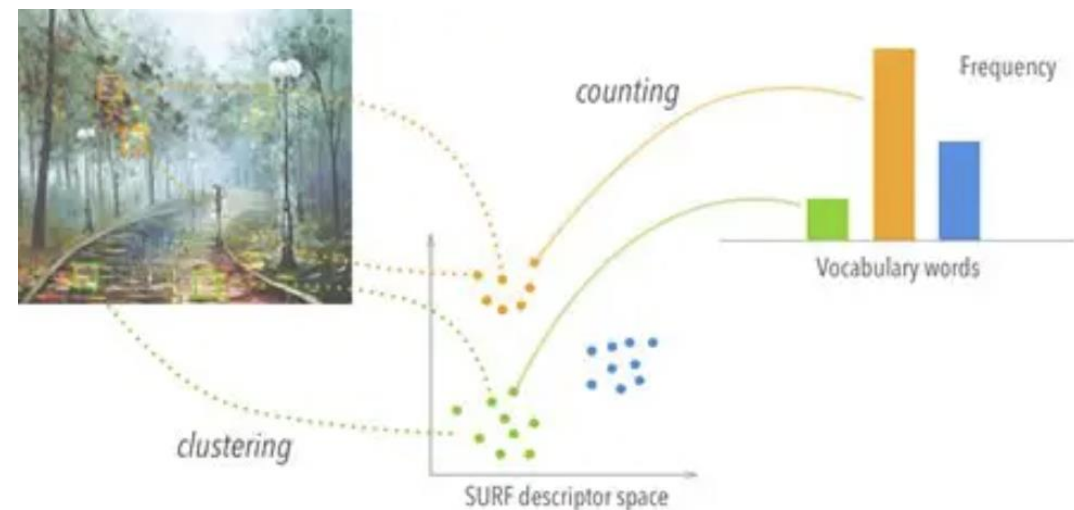
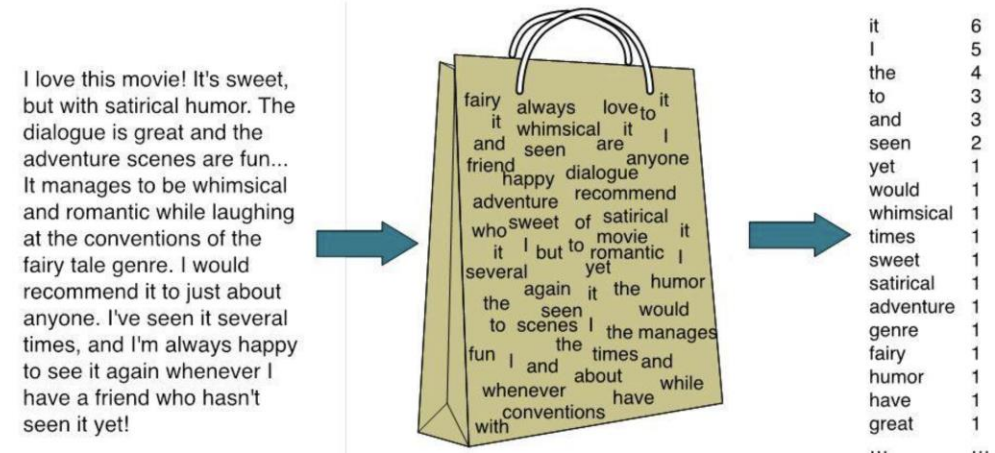
Borrowed from text retrieval. Treat an image as a "bag" of visual words, ignoring their spatial arrangement.

Pipeline:

- **Feature Extraction:** Detect and describe keypoints (e.g., with SIFT).
- **Vocabulary Building:** Cluster all descriptors from the training dataset to create a "visual vocabulary."
- **Quantization:** Assign each new feature to its nearest visual word.
- **Image Representation:** Create a histogram of visual word frequencies for each image.

Matching: Compare histograms using a distance metric (e.g., L1, L2). Fast and scalable!

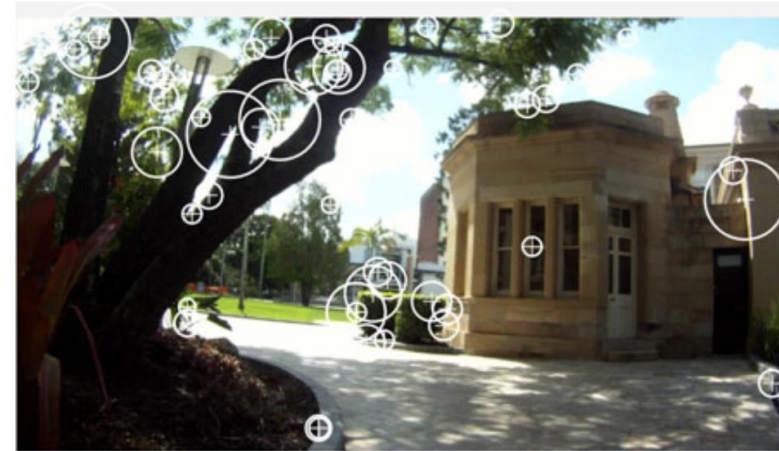
The Bag of Words Representation



Feature Detectors & Descriptors

- **SIFT (Scale-Invariant Feature Transform):** Robust to scale, rotation, and illumination. Computationally heavy.
- **SURF (Speeded Up Robust Features):** Faster approximation of SIFT.
- **ORB (Oriented FAST and Rotated BRIEF):** Fast, binary descriptor. Good for real-time systems. Less robust than SIFT.

These provide the "words" for the BoW model.



(a)



(b)

Deep Learning Based Loop-Closing

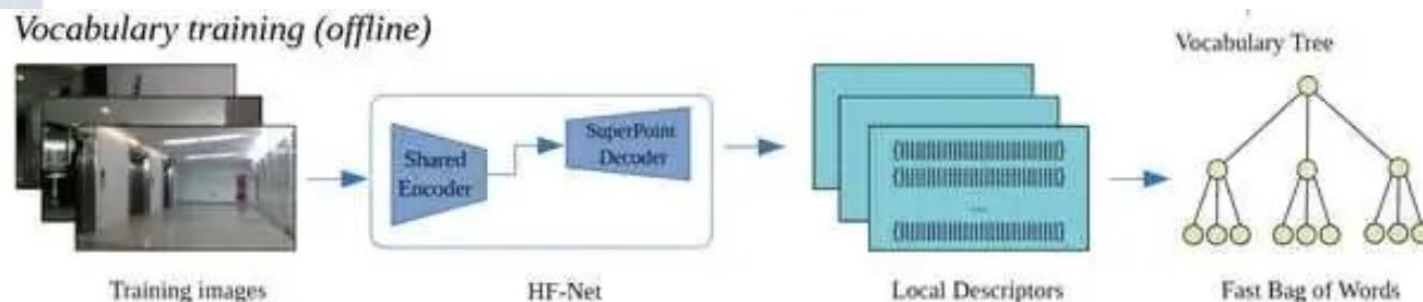
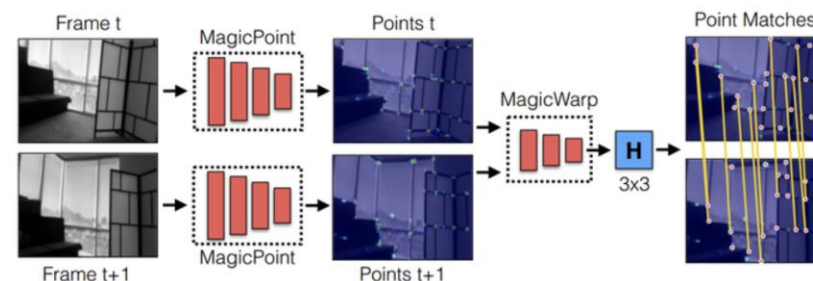
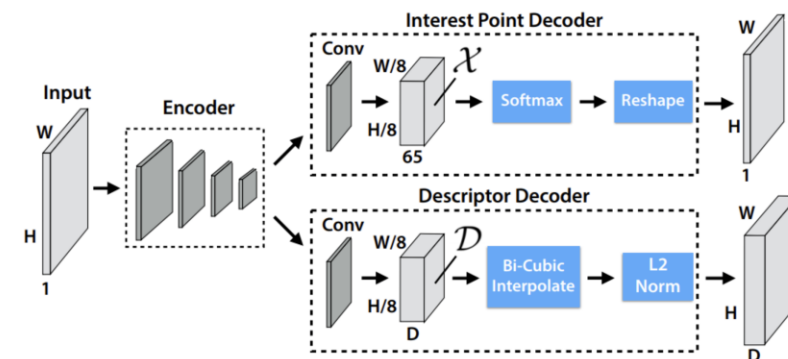
Handcrafted features struggle with severe appearance changes.

The Power of CNNs: Convolutional Neural Networks can learn powerful, condition-invariant features directly from data.

How? Use a pre-trained CNN (e.g., on ImageNet) as a feature extractor.

Global Descriptors: Use the activations from a fully connected layer as a single vector representing the entire image.

Advantage: More robust to viewpoint and condition changes.



The Mapping Problem

Input: sensor data (images, LiDAR, IMU)

Output: spatial representation of the environment

Challenges: dynamic objects, large-scale drift, scale ambiguity, real-time constraints

$$M=f(Z,X)$$

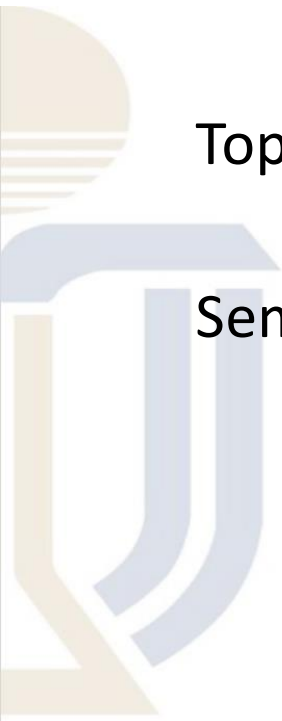
where Z are observations and X are poses.

2D vs. 3D Mapping

2D maps: for ground robots, indoor navigation

3D maps: essential for autonomous driving and drones

Types of Maps



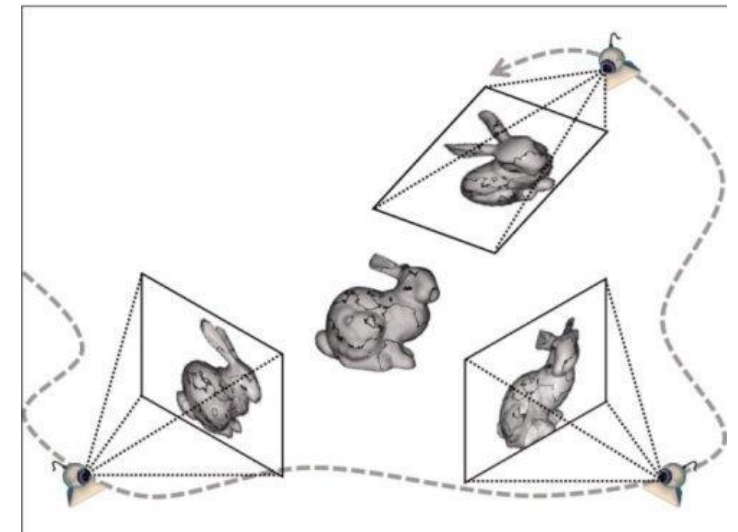
Map Type	Description	Examples
Metric	Precise geometric representation	Point cloud, voxel grid
Topological	Connectivity between places	Graph-based maps
Semantic	Adds meaning to structures	Road, building, pedestrian

Structure-from-Motion

Structure from Motion (SfM) reconstructs the 3D structure of a scene from a sequence of 2D images captured over time or from different viewpoints, without requiring prior camera calibration.

By establishing geometric correspondences between image points, SfM jointly estimates camera parameters and 3D scene geometry.

Corresponding points can be detected using feature descriptors such as SIFT or SURF, and tracked across frames using optical flow methods such as the Lucas–Kanade algorithm.

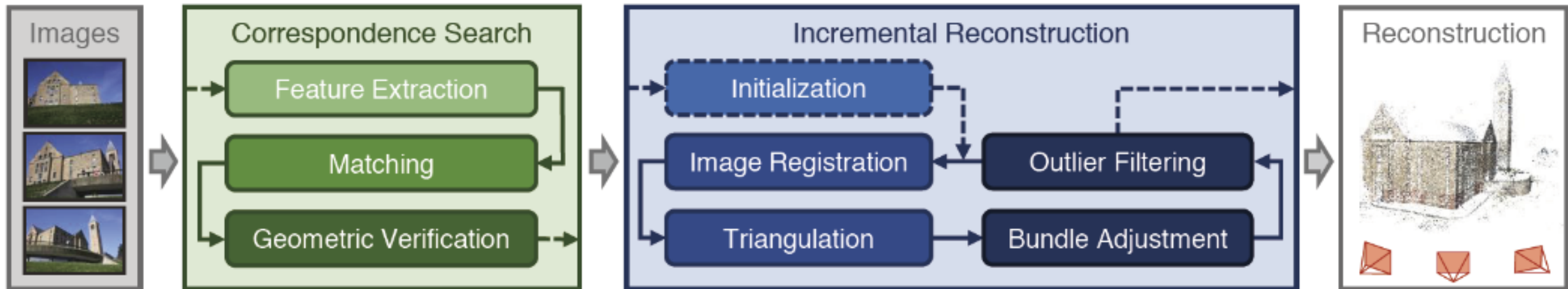


Structure-from-Motion

Colmap: Structure-from-Motion Revisited (2016)

SfM reconstructs 3D scene structure and camera poses from a set of 2D images.
The process can be divided into three main stages:

1. Correspondence Search
2. Feature Extraction & Matching
3. Incremental Reconstruction



Scene Representation

Definition: An internal model describing the structure, semantics, and dynamics of the environment.

Represents geometry, topology, and meaning.

Must support localization, obstacle avoidance, and decision-making.

Desired Properties:

- Accuracy: Faithful geometric reconstruction
- Compactness: Efficient memory use
- Consistency: Coherent over time
- Interpretability: Supports semantic reasoning
- Real-time update: Suitable for dynamic scenes

Scene Representation

Geometric representations describe the **spatial structure** of the scene.

Common forms:

1. Point clouds
2. Meshes
3. Voxel grids
4. Signed distance fields (SDFs)
5. Neural Scene Representations

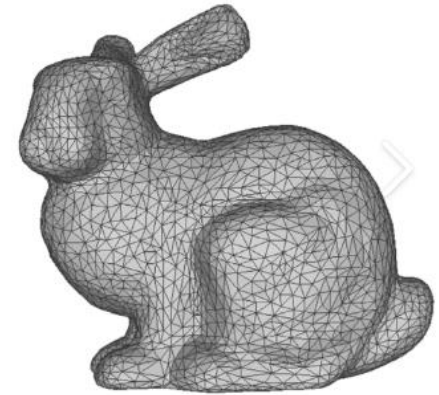
Point cloud



Voxel



Polygon mesh



Point Clouds

- Collection of 3D points sampled from scene surfaces.
- Directly obtained from LiDAR or stereo triangulation.
- Pros: Simple, flexible, accurate.
- Cons: Unstructured, memory-intensive, lacks topology.

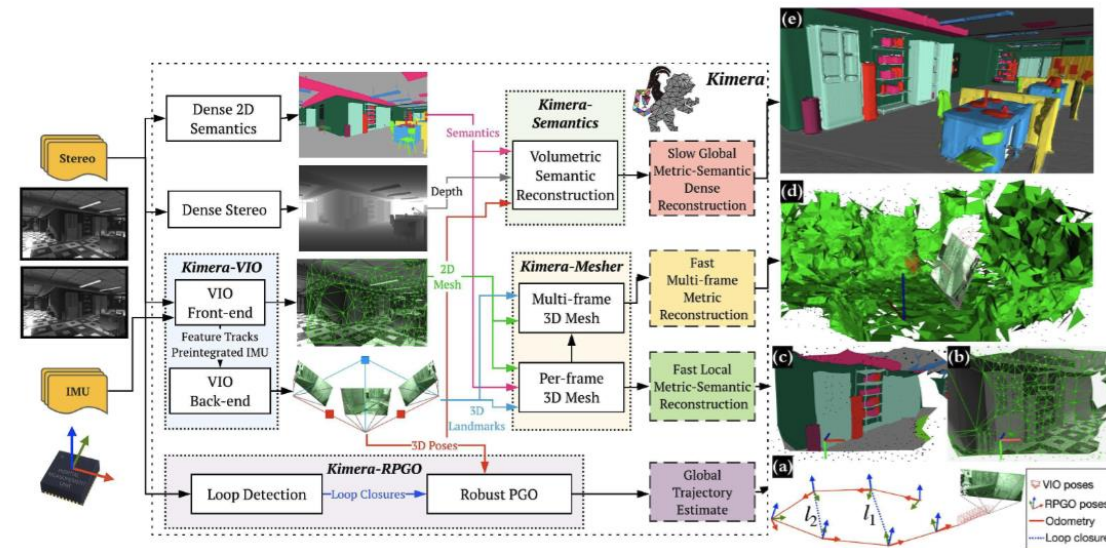
Scene Representation

Mesh-Based Representations

- Define surfaces using **vertices**, **edges**, and **faces**.
- Generated from point clouds via **Delaunay triangulation** or **Poisson reconstruction**.
- Pros: Compact, visually interpretable.
- Cons: Difficult to update dynamically; limited scalability.

Volumetric Representations

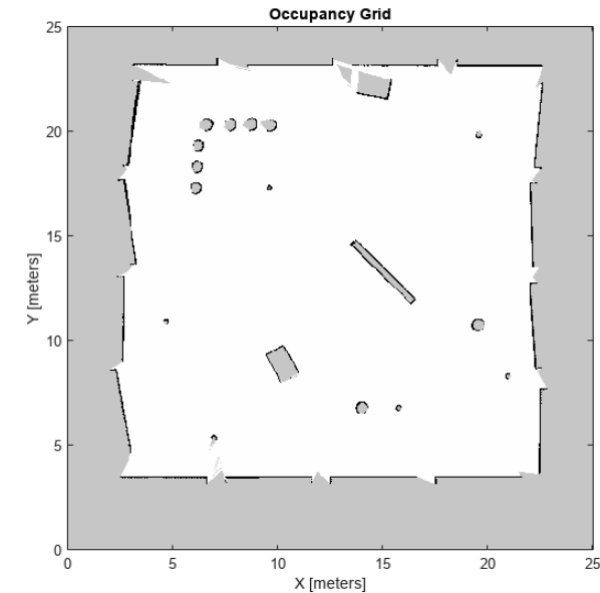
- Environment represented as voxel grid or **Signed Distance Field (SDF)**.
- Used in **KinectFusion**, **ElasticFusion**, **TSDF-SLAM**.
- Each voxel stores occupancy or distance to nearest surface.
- Enables continuous surface extraction via zero-crossing.



The Mapping Problem

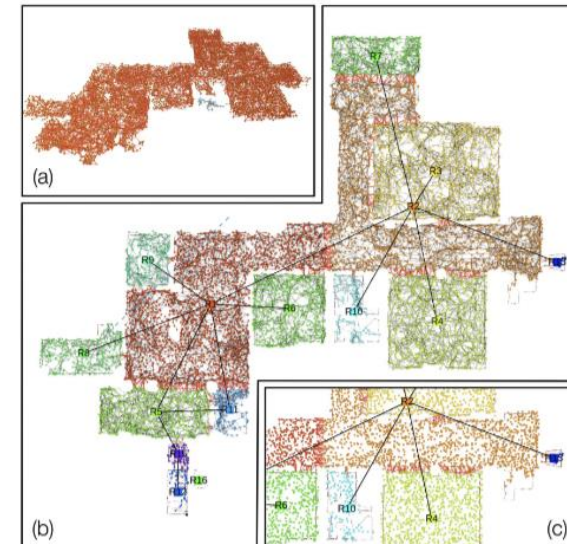
Occupancy Grids

- Divide space into discrete cells; each cell stores occupancy probability.
- Key in 2D/3D mapping and path planning.
- Update rule: Bayesian fusion of sensor measurements.
- Used in autonomous driving for free-space estimation.



Topological Maps

- Represent scene as a **graph** of nodes (places) and edges (connectivity).
- Compact representation suitable for large-scale navigation.
- Example: Pose graph in SLAM, lane graph in HD maps.

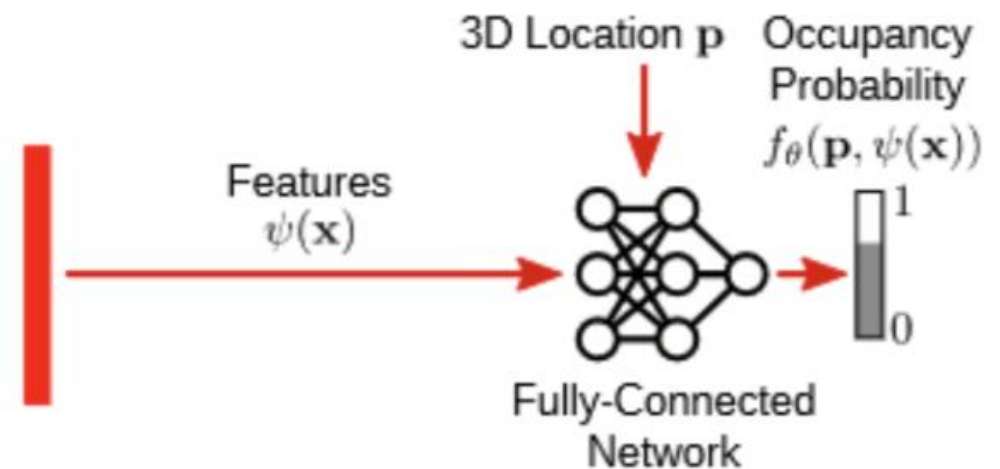
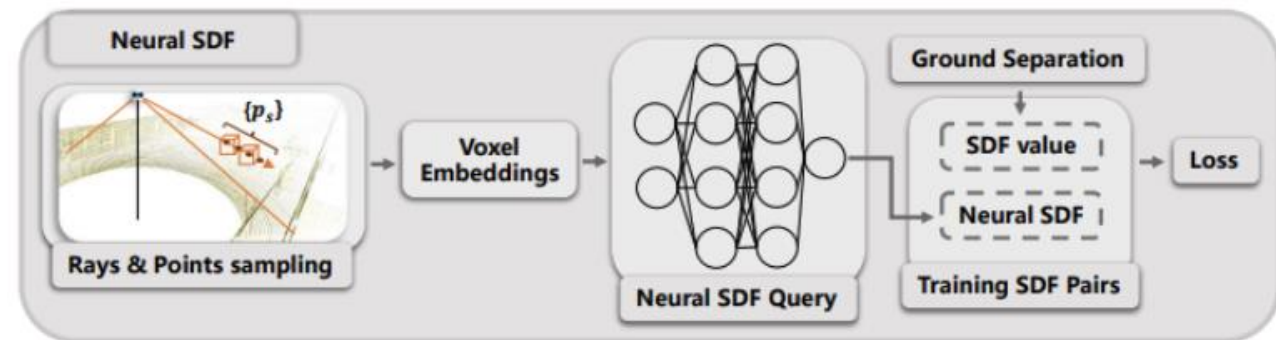


Neural Implicit Representations

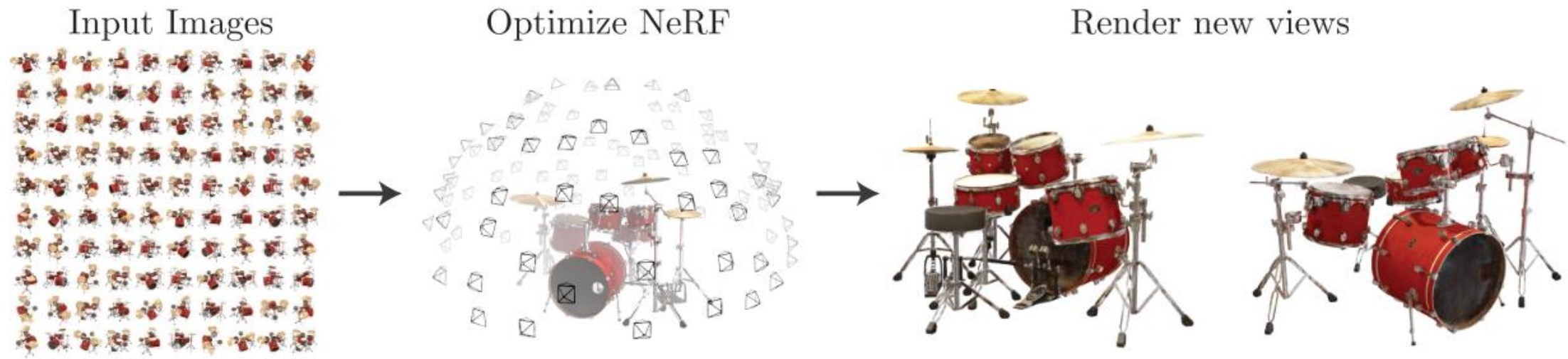
Encode 3D scenes as continuous neural fields.
Implicit function

$f_{\theta}(\mathbf{x}) \rightarrow$ color, density, occupancy.

Smooth, memory-efficient, and differentiable.



Neural Rendering



NeRF: Represent scenes as continuous radiance and density fields.

Learn by minimizing photometric loss across views.

Enable high-fidelity rendering from new viewpoints.

$$F_{\theta} : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$$

where

- $\mathbf{x}$: 3D location
- $\mathbf{d}$: viewing direction
- $\mathbf{c}$: RGB color
- σ : volume density

Neural Rendering

1. Volume Rendering Principle

Each pixel's color is computed by integrating the emitted radiance and transmittance along the camera ray:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) c(\mathbf{r}(t), \mathbf{d}) dt$$

2. Training Objective

- Rendered color is compared with ground truth image pixel color

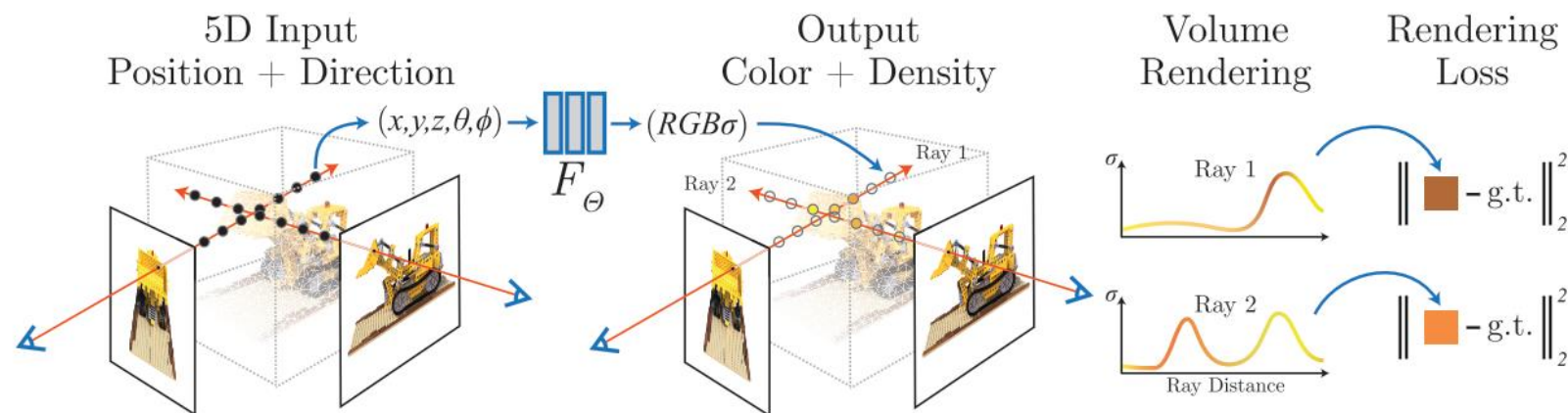
- Loss:

$$\mathcal{L} = \sum_{\mathbf{r}} \|C(\mathbf{r}) - C^*(\mathbf{r})\|^2$$

3. Optimization

- Train MLP parameters θ using multiple images with known camera poses.
- The network learns to encode scene geometry (via density) and appearance (via color).

$$T(t) = \exp \left(- \int_{t_n}^t \sigma(\mathbf{r}(s)) ds \right)$$



Neural Rendering

Extensions:

NeRF-SLAM (2022): Jointly optimizes camera poses and radiance field → implicit dense mapping.

iMAP, NICE-SLAM: Real-time incremental NeRF-based SLAM systems.

NeRF++ / Instant-NGP: Improve rendering efficiency with spatial decomposition and hash encoding.

Dynamic NeRF (D-NeRF): Models temporal variations for moving scenes.





Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)